CCT College Dublin

Assessment Cover Page

Module Title:	Mobile development
Assessment Title:	Diploma in Applied Software Development
Lecturer Name:	Kayoum khbuli
Student Full Name:	James Scott
Student Number:	Sba23056
Assessment Due Date:	Thursday, 4 July 2024, 11:59 PM
Date of Submission:	Monday, 1 July 2024

Declaration

By submitting this assessment, I confirm that I have read the CCT policy on Academic Misconduct and understand the implications of submitting work that is not my own or does not appropriately reference material taken from a third party or other source. I declare it to be my own work and that all material from third parties has been appropriately referenced. I further confirm that this work has not previously been submitted for assessment by myself or someone else in CCT College Dublin or any other higher education institution.

Mobile development

Student Name: James Scott

Student Number: Sba23056

Date: 01 Jul 2024

TABLE OF CONTENTS

1.0. SUMMARY

This report will outline the optimal approach to integrating data management, sensors, location services, maps, and networking functionalities into a fleet management mobile application. The application aims to track and manage a fleet of vehicles within a company in real-time, ensuring efficiency, robustness, and a seamless user experience. Key areas of focus include data management strategies using shared preferences, SQLite databases for internal storage, and Firebase for external storage; the utilization of various sensors for tracking and monitoring; the integration of location-based services and maps using the Google Maps API; and the management of network connectivity through Wi-Fi and mobile data.

Additionally, the report emphasizes the importance of using accelerometers and gyroscopes to monitor driving behaviour and vehicle movements, which can help improve safety for staff and general public and optimize routes ensuring fast and effective real time location. It also highlights best practices for managing sensor data to conserve battery life and ensure efficient performance.

Furthermore, the integration of real-time data synchronization through Firebase ensures that critical information is always up to date across all devices. By implementing effective networking strategies, such as using RESTful APIs and optimizing data payloads, the application can maintain reliable communication with backend servers, even in varying network conditions. These combined efforts aim to deliver a robust and user-friendly fleet management solution.

2.0. INTRODUCTION

In the contemporary business landscape, the ability to track and manage a fleet of vehicles in real-time is essential for operational efficiency and productivity. Mobile applications designed for fleet management must integrate various functionalities such as data management, sensors, location services, maps, and networking to deliver a seamless and effective user experience. This report explores the best practices and strategies for integrating these functionalities into a fleet management system, ensuring the application is both efficient and robust.

The integration of data management systems, such as shared preferences for user settings, SQLite for local data storage, and Firebase for cloud-based storage, is crucial for maintaining data integrity and accessibility. These systems allow for efficient handling of user preferences, secure storage of

sensitive information, and real-time synchronization of critical data across multiple devices. By leveraging these technologies, the application can provide a reliable and consistent user experience.

Moreover, the use of sensors like accelerometers and gyroscopes, along with location services and mapping tools, enhances the application's ability to monitor and manage the fleet effectively. These technologies enable real-time tracking of vehicle movements, detection of driving behaviours, and optimization of routes. Coupled with robust networking strategies to manage Wi-Fi and mobile data connectivity, the application ensures continuous and reliable communication with backend servers, even in challenging network conditions. This comprehensive approach ensures that the fleet management system is both powerful and user-friendly.

3.0. FLEET MANAGEMENT MOBILE APPLICATION

In today's fast-paced and interconnected world, fleet management has become a crucial aspect of many businesses, enabling efficient tracking and management of vehicle fleets. A robust fleet management mobile application is essential for streamlining operations, enhancing productivity, and ensuring the security and maintenance of vehicles. This section explores the key components of data management within such an application, focusing on the appropriate use of shared preferences, internal and external storage, SQLite for managing complex data locally, and Firebase for cloud-based data synchronization and storage.

3.1. Data Management

Effective data management is critical for the performance and security of a fleet management application. The following sections detail the appropriate use of shared preferences, SQLite for internal storage, and Firebase for external storage.

3.1.1. Files & Shared Preferences

Shared preferences are best suited for storing simple data like user settings and preferences, providing quick and easy access. Shared preference use Case can be Saving user login states, application settings, and last viewed positions.

Internal Storage is ideal for storing sensitive data securely within the application's private directory. External Storage is used for storing larger files that need to be shared across applications or accessed by the user. Refer to **Table 1** for more details.

Table 1 Advantages and Disadvantages of Internal and External Storage.

Advantages of Internal Storage	**Advantages of External Storage**
• Enhanced security and data privacy • Suitable for storing credentials and configuration files.	• More storage space and accessibility for large files like logs and reports.
Disadvantages of Internal Storage	**Disadvantages of External Storage**
• Limited storage opportunity • More difficult to upgrade or expand.	• Potential security risks if not properly managed • May require additional hardware and maintenance.

3.1.2. Mobile Storage (SQLite and Firebase)

SQLite is a powerful and lightweight database engine embedded within the mobile application, ideal for managing complex data through structured queries. It supports CRUD (Create, Read, Update, Delete) operations, indexing, and transactions, ensuring data integrity and performance. The advantages of Mobile Storage: SQLite are:

- Supports complex queries and transactions.
- Ensures data consistency through atomic, consistent, isolated, and durable (ACID) properties.
- Efficient for handling large datasets and relational data.

Firebase provides a cloud-based database solution for real-time data synchronization and storage. It is particularly useful for data that needs to be accessible across multiple devices and for collaborative features. The advantages of Mobile Storage: Firebase are:

- Real-time data synchronization across devices.
- Cloud-based storage that scales automatically.
- Secure data transfer and storage with built-in authentication and access controls.

Best Practices for the use of SQLite and Firebase data storage:

- Use SQLite for local data storage and operations that require low latency.
- Synchronize critical data with Firebase to ensure consistency across devices.
- Implement proper error handling and data validation to maintain data integrity.
- Regularly perform database maintenance such as vacuuming and analyzing in SQLite.

In addition to these strategies, it is essential to implement robust data encryption methods to protect sensitive information both at rest and in transit. This includes using encryption libraries for SQLite databases and ensuring that data transmitted to and from Firebase is encrypted using HTTPS. By prioritizing data security, the application can safeguard user information and maintain compliance with data protection regulations.

Furthermore, implementing a comprehensive backup strategy is crucial for data recovery and business continuity. Regularly scheduled backups of SQLite databases and Firebase data can prevent data loss in case of unexpected failures or cyber-attacks. Automated backup solutions and periodic testing of backup restoration processes ensure that the application can quickly recover from data loss incidents, thereby minimizing downtime and maintaining operational efficiency.

3.2. Sensors, Location & Maps

Integrating sensors, location services, and maps is essential for tracking and managing the fleet effectively.

3.2.1. Measurements of Physical Environment

Various sensors such as the accelerometer and gyroscope can enhance the functionality of a fleet management application.

- **Accelerometer:** Measures the acceleration of the vehicle, which can be used for detecting motion, sudden stops, or crashes.
- **Gyroscope:** Measures the orientation of the vehicle, providing data on angular velocity, which can be used for tracking driving behaviour.

Accelerometer and Gyroscope use cases:

- Monitoring driving patterns and behaviours to improve safety.
- Detecting harsh braking, acceleration, or collisions.
- Analysing vehicle movements to optimize routes and fuel efficiency.

3.2.2. Sensor Manager and Sensor Event

The Sensor Manager class in Android provides an interface to access and manage sensors. It allows the application to register listeners for sensor events and obtain sensor data efficiently.

Best Practices:

- Register for sensor updates at the minimal required frequency to conserve battery.
- Use batch processing for sensor data to reduce wakeups and save power.
- Unregister sensor listeners when not needed to prevent unnecessary battery drain.

3.2.3. Location and Maps

Integrating location-based services and maps is crucial for real-time tracking and management of the fleet.

- **Google Maps API:** Provides powerful tools for displaying maps, plotting routes, and visualizing the location of all fleet vehicles.
- **Fused Location Provider API:** Offers a high-level framework for accessing the device's location with improved accuracy and lower power consumption.

Strategies for Optimization:

- Use a balanced power mode to get the best trade-off between accuracy and battery life.
- Implement geofencing to trigger specific actions when vehicles enter or exit defined areas.
- Cache map data for offline use to improve performance and reliability.

In addition to these strategies, leveraging advanced mapping features such as traffic data and route optimization can significantly enhance the efficiency of fleet operations. By integrating real-time traffic updates, the application can dynamically adjust routes to avoid congestion, thereby reducing travel time and fuel consumption. Route optimization algorithms can further

ensure that the most efficient paths are chosen, taking into account factors such as distance, traffic conditions, and delivery schedules.

Moreover, incorporating geospatial analytics can provide deeper insights into fleet operations. By analysing location data over time, fleet managers can identify patterns and trends that may indicate areas for improvement. For example, frequent stops or deviations from planned routes can be investigated to understand underlying issues such as driver behaviour or road conditions. These insights can inform strategic decisions, leading to more efficient and cost-effective fleet management.

3.2.4. GDPR Compliance

When integrating location-based services, it is crucial to comply with the General Data Protection Regulation (GDPR) to protect the privacy of employees. GDPR mandates that personal data, including location information, must be processed lawfully, transparently, and for a specific purpose. Employers must obtain explicit consent from employees before tracking their location and must inform them about the extent and purpose of the tracking.

To ensure compliance, the application should include features that allow employees to control their location tracking settings, such as enabling or disabling tracking during private hours. Employers should also implement strict data access controls and ensure that location data is only accessible to authorized personnel. Additionally, data retention policies should be established to ensure that location data is not stored longer than necessary. By adhering to these GDPR protocols, the application can respect employee privacy while still providing valuable fleet management capabilities.

3.3. Networking

Efficient networking is vital for ensuring that the fleet management application remains responsive and reliable, especially when dealing with real-time data.

3.3.1. Wi-Fi Interaction

Managing Wi-Fi connectivity is important to ensure seamless data transfer and maintain the application’s responsiveness.

Best Practices:

- Enable or disable Wi-Fi based on network availability and user settings.
- Check connectivity status regularly and handle network state changes gracefully.
- Implement fallbacks to mobile data if Wi-Fi is unavailable.

3.3.2. Mobile Data Management

Effective strategies for managing mobile data are necessary to handle different types of network requests and ensure data synchronization.

Network Requests:

- Use RESTful APIs for communication with backend servers.
- Implement retry mechanisms for failed requests to handle intermittent connectivity issues.
- Optimize data payloads to reduce bandwidth usage.

Data Synchronization:

- Use background services to synchronize data periodically or based on specific triggers.
- Implement local caching to store data temporarily and reduce the need for constant network access.
- Utilize push notifications to inform the application of real-time updates from the server.

In addition to these strategies, it is essential to prioritize network security to protect sensitive data transmitted between the application and backend servers. Implementing secure communication protocols such as HTTPS ensures that data is encrypted during transmission, preventing unauthorized access and data breaches. Regularly updating the application and its dependencies can also mitigate vulnerabilities and enhance overall security. By maintaining a secure network environment, the application can safeguard user data and maintain trust.

Moreover, optimizing network usage can significantly improve the application's performance and user experience. Techniques such as data compression and efficient data serialization can reduce the amount of data transmitted over the network, leading to faster response times and lower data consumption. Additionally, implementing intelligent data synchronization strategies, such as

delta synchronization, can minimize the amount of data transferred by only sending changes rather than the entire dataset. These optimizations not only enhance the application's efficiency but also reduce operational costs associated with data usage.

4.0. CONCLUSION

Integrating data management, sensors, location services, maps, and networking functionalities into a fleet management mobile application involves a careful balance of performance, efficiency, and user experience. By utilizing shared preferences, SQLite for secure local storage, and Firebase for real-time cloud storage, the application can manage data effectively. Leveraging sensors, location services, and maps ensures accurate and real-time tracking of the fleet. Efficient networking practices, including managing Wi-Fi and mobile data, ensure the application remains responsive and reliable. By following these strategies, the fleet management system can provide a robust and seamless experience for users.

The integration of advanced sensor technologies, such as accelerometers and gyroscopes, allows for comprehensive monitoring of vehicle dynamics and driver behaviour. This data can be used to enhance safety protocols, optimize routes, and improve overall fleet efficiency. Additionally, the use of geospatial analytics and real-time traffic data can further refine route planning and reduce operational costs. These technological advancements not only improve the functionality of the fleet management application but also contribute to a safer and more efficient fleet operation.

Furthermore, ensuring compliance with data protection regulations, such as GDPR, is crucial for maintaining user trust and protecting employee privacy. Implementing robust data encryption methods, secure communication protocols, and strict data access controls can safeguard sensitive information. Additionally, providing employees with control over their location tracking settings and establishing clear data retention policies can help balance the need for operational efficiency with respect for individual privacy. By adhering to these best practices, the fleet management application can achieve a high level of security and compliance.

In conclusion, the successful integration of data management, sensor technologies, location services, maps, and networking functionalities into a fleet management mobile application requires a holistic approach that prioritizes performance, efficiency, and user experience. By

leveraging the strengths of SQLite and Firebase for data management, utilizing advanced sensors for monitoring, and implementing robust networking strategies, the application can deliver real-time, accurate, and reliable fleet management capabilities. Additionally, ensuring compliance with data protection regulations and optimizing network usage can further enhance the application's security and efficiency. By following these comprehensive strategies, fleet managers can achieve a seamless and effective fleet management solution that meets the demands of modern business operations

5.0. REFERENCES

I. Google Developers, 2023. Google Maps Platform. [online] Available at: https://developers.google.com/maps [Accessed 14 June 2024].

II. Android Developers, 2023. Sensor Manager. [online] Available at: https://developer.android.com/reference/android/hardware/SensorManager [Accessed 14 June 2024].

III. SQLite, 2023. About SQLite. [online] Available at: https://www.sqlite.org/about.html [Accessed 14 June 2024].

IV. Firebase, 2023. Firebase Realtime Database. [online] Available at: https://firebase.google.com/products/realtime-database [Accessed 14 June 2024].

V. European Union, 2018. General Data Protection Regulation (GDPR). [online] Available at: https://gdpr.eu/ [Accessed 14 June 2024].

VI. Google Developers, 2023. Fused Location Provider API. [online] Available at: https://developers.google.com/location-context/fused-location-provider [Accessed 14 June 2024].

VII. OWASP, 2023. Transport Layer Protection Cheat Sheet. [online] Available at: https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html [Accessed 14 June 2024].

VIII. Android Developers, 2023. Data Backup Overview. [online] Available at: https://developer.android.com/guide/topics/data/backup [Accessed 14 June 2024].

APPENDICES

Figure 1: Fleet Management

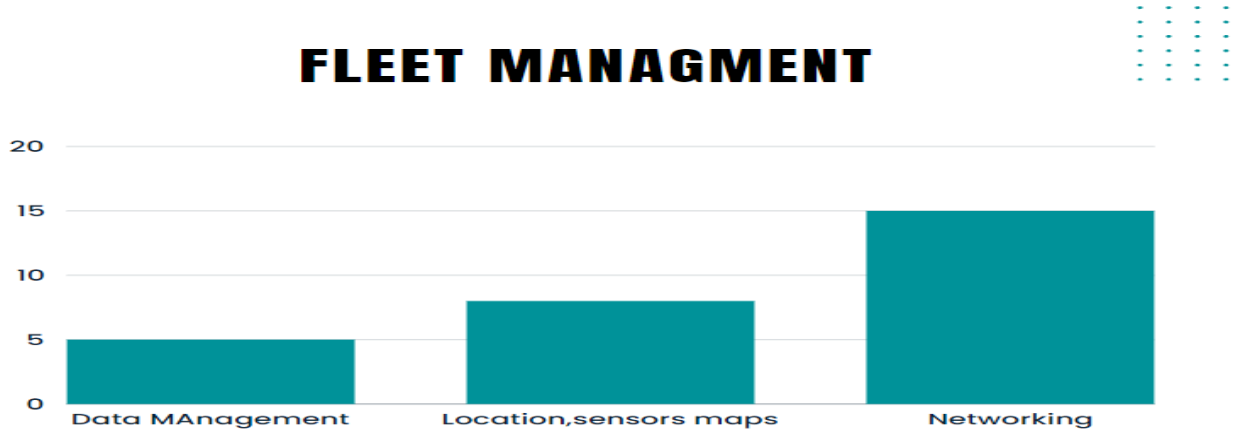


Figure 2: Data Management

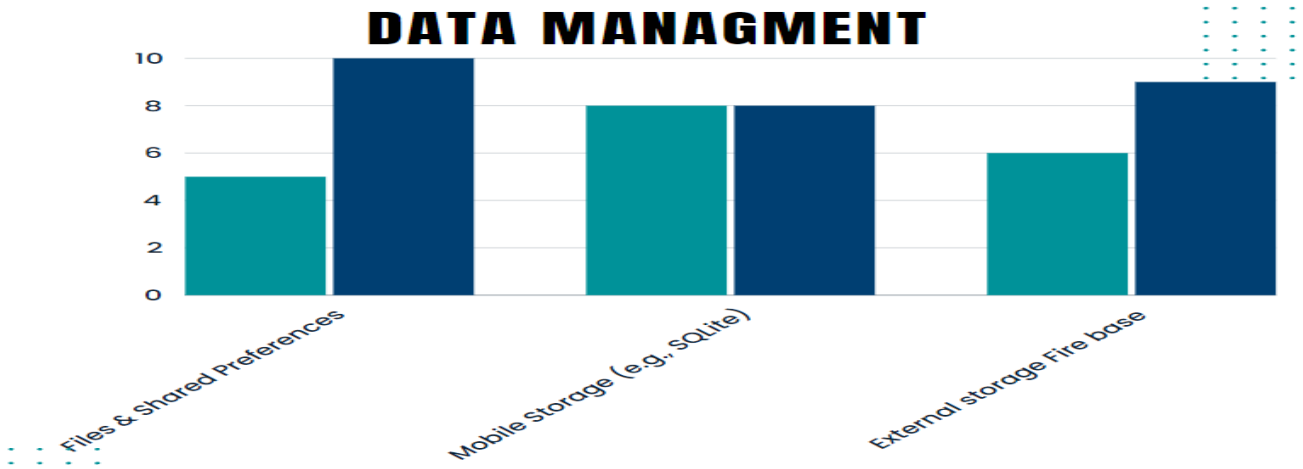


Figure 3: Sensor, Location and Maps

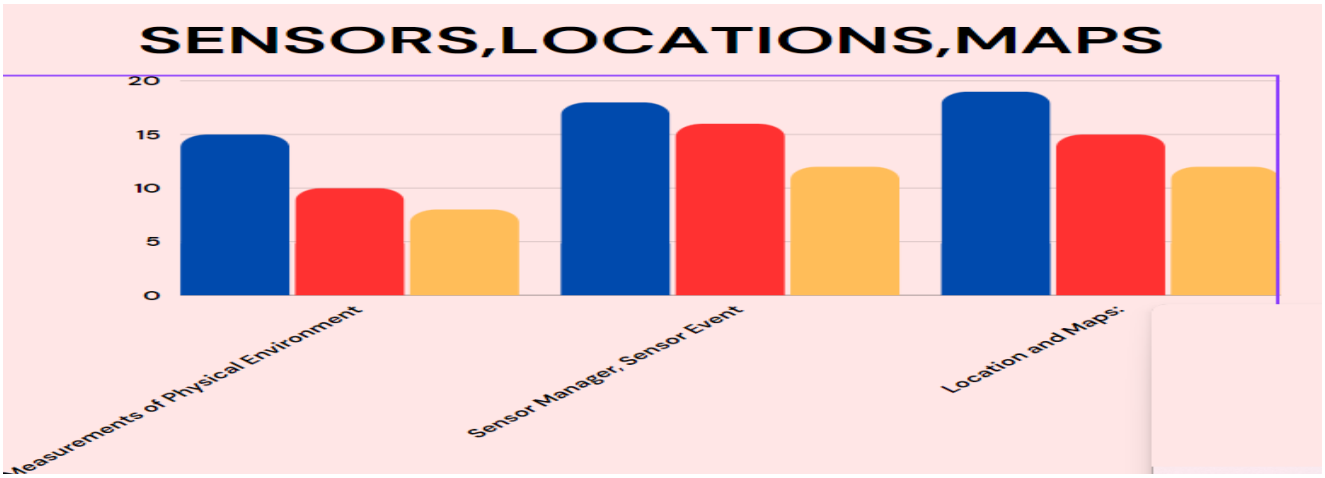
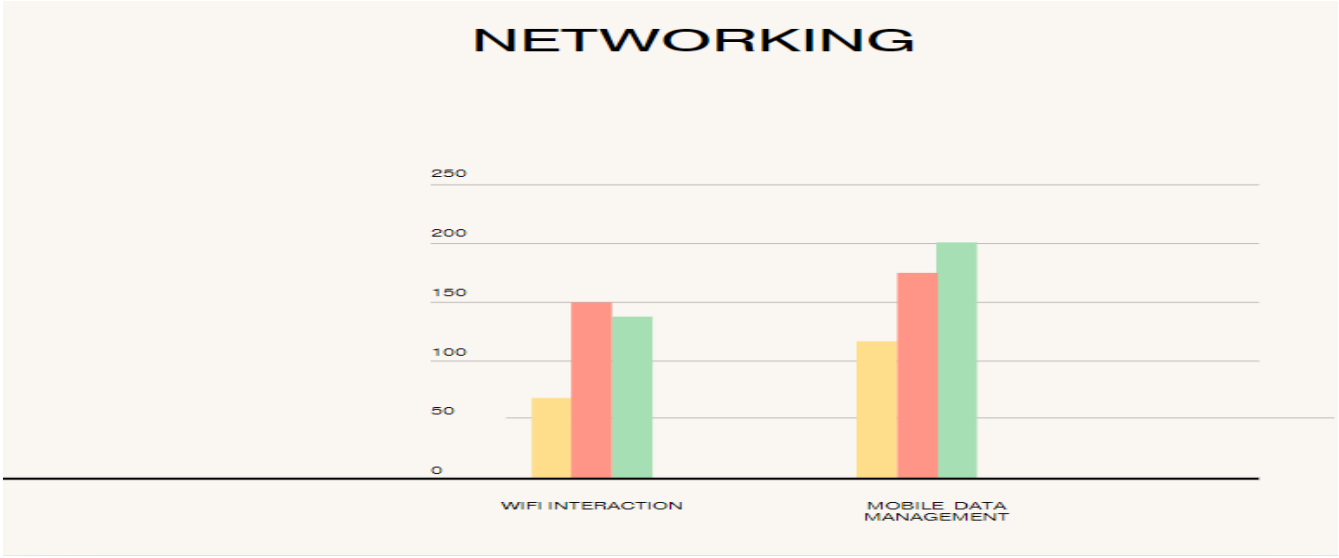


Figure 4: Networking



Code Example 1: Code example for reading user information

```
 import android.content.Context;

import java.io.FileInputStream;

import java.io.IOException;

public class InternalStorageHelper {

    private Context context;

    public InternalStorageHelper(Context context) {

        this.context = context;

    }

    public String readUserData() {

        String filename = "user_data.txt";

        FileInputStream fis = null;

        StringBuilder stringBuilder = new StringBuilder();

        try {

            // Open the file for reading

            fis = context.openFileInput(filename);

            int character;

            while ((character = fis.read()) != -1) {

                stringBuilder.append((char) character);

            }

            System.out.println("User data read successfully");
```

```
        } catch (IOException e) {
            e.printStackTrace();
            System.err.println("Error reading user data: " + e.getMessage());
        } finally {
            if (fis != null) {
                try {
                    fis.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        }

        return stringBuilder.toString();
    }
}
```

The code I have generated has not yet been tested and may require further integration and testing to ensure its functionality and reliability.

Code Example 2: Code example for using maps location

```
import android.os.Bundle;

import androidx.fragment.app.FragmentActivity;

import com.google.android.gms.maps.CameraUpdateFactory;

import com.google.android.gms.maps.GoogleMap;

import com.google.android.gms.maps.OnMapReadyCallback;

import com.google.android.gms.maps.SupportMapFragment;

import com.google.android.gms.maps.model.LatLng;

import com.google.android.gms.maps.model.MarkerOptions;

public class MapsActivity extends FragmentActivity implements OnMapReadyCallback {

   private GoogleMap mMap;

   @Override

   protected void onCreate(Bundle savedInstanceState) {

      super.onCreate(savedInstanceState);

      setContentView(R.layout.activity_maps);

      // Obtain the SupportMapFragment and get notified when the map is ready to be used.

      SupportMapFragment mapFragment = (SupportMapFragment)
getSupportFragmentManager()

            .findFragmentById(R.id.map);

      mapFragment.getMapAsync(this);
```

```
    }

    @Override

    public void onMapReady(GoogleMap googleMap) {

        mMap = googleMap;

        // Add a marker inireland and move the camera

        LatLng ireland = new LatLng(-34, 151);

        mMap.addMarker(new MarkerOptions().position(ireland).title("Generated marker"));

        mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(ireland, 10));

    }

}
```

The code I have generated has not yet been tested and may require further integration and testing to ensure its functionality and reliability.

Figure 5: Main screen image

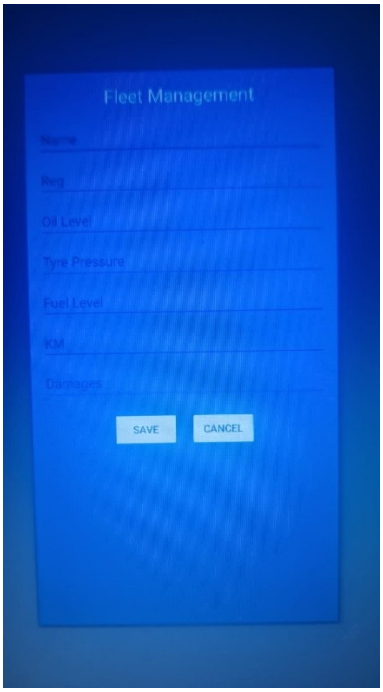


Figure 6: Login screen

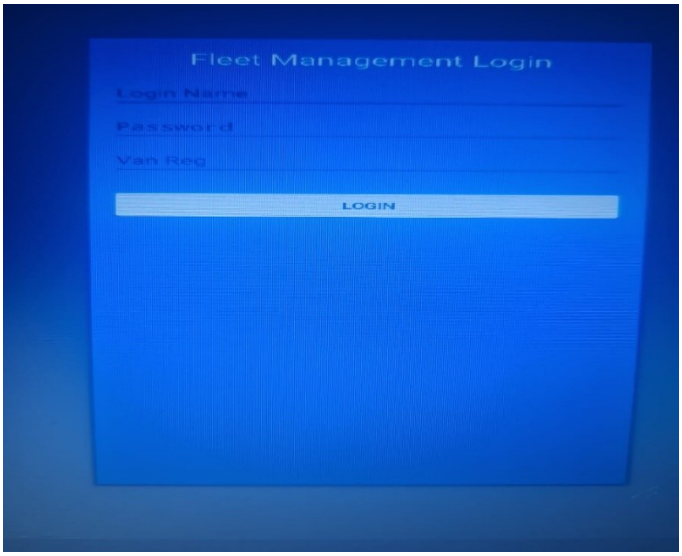


Figure 7: Map location